Assignment # 1

Student Name: Subhana Bushra

Registration Number: FA25-BDS-046

Class/Section: BDS-1A

Semester: 2nd

Department: Computer Science

Program: Data Science

Instructor: Sir Rizwan Rashid

Course: Object Oriented Programming

Lab 1- Problem Solving in Object Oriented Paradigm

Task 1 – A Student is an object in a university management System. Analyze the concept and identify the data members that a Student class should have. Also analyze the behavior of student in a university management System and identify the methods that should be included in Student class.

Source Code

```
import java.util.Scanner;
class Student {

    int studentId;
    String name;
    int age;
    String program;
    int semester;
    String[] courses = new String[5];

    void registerCourses() {
        for (int i = 0; i < 5; i++) {
            System.out.print("Enter course " + (i + 1) + ": ");
            Scanner input = new Scanner(System.in);
            courses[i] = input.nextLine();
        }
    }

    void displayStudent() {
        System.out.println("Student ID: " + studentId);
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Program: " + program);
        System.out.println("Semester: " + semester);

        System.out.println("Registered Courses:");
        for (int i = 0; i < 5; i++) {
            System.out.println(courses[i]);
        }
    }
}

public class studentManagementSystemLab1Task1 {
```

```java
    public static void main(String[] args) {

        java.util.Scanner input = new java.util.Scanner(System.in);

        Student s1 = new Student();

        System.out.print("Enter Student ID: ");
        s1.studentId = input.nextInt();
        input.nextLine();

        System.out.print("Enter Name: ");
        s1.name = input.nextLine();

        System.out.print("Enter Age: ");
        s1.age = input.nextInt();
        input.nextLine();

        System.out.print("Enter Program: ");
        s1.program = input.nextLine();

        System.out.print("Enter Semester: ");
        s1.semester = input.nextInt();

        s1.registerCourses();
        s1.displayStudent();
    }
}
```

Output

```
Enter Student ID: 99
Enter Name: subhana
Enter Age: 18
Enter Program: data science
Enter Semester: 2
Enter course 1: english
Enter course 2: oops
Enter course 3: islamiat
Enter course 4: civics
Enter course 5: bioinformatics
Student ID: 99
Name: subhana
Age: 18
Program: data science
Semester: 2
Registered Courses:
english
oops
islamiat
civics
bioinformatics
```

Task 2 – Time is an intangible concept. Analyze the concept and identify the data members and methods that should be included in Time class.

Source Code

```
class Time{
    public int hours;
    public int minutes;
    public int seconds;
    public void display(){
    System.out.println("The time is:" + hours + ":" + minutes + ":" + seconds);
}
}
public class TimeDisplayLab1Task2{
    public static void main (String [] args){
        Time t1 = new Time();
        Time t2 = new Time();
        t1.hours = 10;
        t1.minutes = 2;
        t1.seconds = 3;
        t1.display();
        t2.hours = 3;
        t2.minutes = 10;
        t2.seconds = 39;
        t2.display();
    }
}
```

Output

```
The time is:10:2:3
The time is:3:10:39
```

Task 3 – Car is an object that helps us in transportation. Analyze the concept and identify the data members and methods that should be included in Car class.

Source Code

```
class Car {

    public String brand;
    public int speed;
    public int price;

    public void show() {
        System.out.println("Brand of the car is: " + brand);
        System.out.println("Speed of the car is: " + speed + " km/h");
        System.out.println("Price of the car is: " + price);
        System.out.println();
    }

    public void speedUp(int increase) {
        speed = speed + increase;
    }

    public void speedDown(int decrease) {
        speed = speed - decrease;

        if (speed < 0) {
            speed = 0;
        }
    }
}

public class CarConceptLab1Task3 {
    public static void main(String[] args) {

        Car c1 = new Car();
        Car c2 = new Car();

        c1.brand = "Toyota";
        c1.speed = 120;
        c1.price = 900000;

        c2.brand = "Honda";
        c2.speed = 140;
        c2.price = 1000000;
```

```
        c1.speedUp(20);
        c1.speedDown(50);

        c2.speedUp(10);
        c2.speedDown(30);

        c1.show();
        c2.show();
    }
}
```

Output

```
Brand of the car is: Toyota
Speed of the car is: 90 km/h
Price of the car is: 900000

Brand of the car is: Honda
Speed of the car is: 120 km/h
Price of the car is: 1000000
```

Task 4 – Rectangle is an object that represents a specific shape. Analyze the concept and identify the data members and methods that should be included in Rectangle class.

Source Code

```
class Rectangle {
    public int length;
    public int width;
    public int area() {
        return length * width;
    }
    public int perimeter() {
        return 2 * (length + width);
    }
    public void display() {
        System.out.println("The area of the rectangle is: " + area());
        System.out.println("The perimeter of the rectangle is: " + perimeter());
    }
}
public class RectangleConceptLab1Task4 {
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle();
        r1.length = 9;
        r1.width = 8;
        r1.display();
    }
}
```

Output

```
The area of the rectangle is: 72
The perimeter of the rectangle is: 34
```

Lab 2- Defining classes and objects in JAVA

Task 1 – Create a class circle class with radius as data member. Create two constructors (no argument, and two arguments) and a method to calculate Circumference.

Source Code

```
//Circle as an object
import java.util.Scanner;
class Circle {
    public Scanner sc = new Scanner(System.in);
    public double radius;
    public double pi = 3.14;
    public Circle() { //this one is no arg constructor
        System.out.println("Enter the radius of the circle:");
        radius = sc.nextDouble();
    }
    public Circle(double r) { //paramterized constructor this one
        radius = r;
    }
    public double calculateCircumference() {
        return 2 * pi * radius;
    }
}
public class RunnerLab2Task1 {
    public static void main(String[] args) {
        Circle c1 = new Circle();
        System.out.println("The circumference of the circle is: " +
c1.calculateCircumference());
    }
}
```

Output

```
Enter the radius of the circle:
56
The circumference of the circle is: 351.68
```

Task 2 – Create a class Account class with balance as data member. Create two and methods to withdraw and deposit balance.

Source Code

```
import java.util.Scanner;
class Account{
    private double balance;
    Scanner sc = new Scanner(System.in);
    public Account(){ // no arg constructor
        balance = 0;
    }
    public Account( double b){
        balance = b;
    }
    public void  withdraw( double w){
        System.out.println("Enter the amount withdrawn:");
        w = sc.nextDouble();
        balance = balance - w;
    }
    public void deposit( double d){
        System.out.println("Enter the amount deposited:");
        d = sc.nextInt();
        balance = balance + d;
    }
    public void display(){
        System.out.println("The balance is:" + balance);
    }
    }
    public class RunnerLab2Task2{
        public static void main(String [] args){
            Account a1 = new Account();
            a1.display();
            Account a2 = new Account(1000);
            a2.display();
        }
    }
```

Output

```
The balance is:0.0
The balance is:1000.0
```

Task 3 – Create a class —Distance with two constructors (no argument, and two argument), two data members (feet and inches). Also create display function which displays all data members.

Source Code

```
class Distance{
    private int feet;
    private int inches;

    public Distance(){ //no arg constructor
        feet = 1;
        inches = 2;
    }
    public Distance( int f, int i){
        feet = f;
        inches = i;
    }
    public void display(){
        System.out.println("The distance is : " + feet + "'" + inches);
    }
}
public class RunnerLab2Task3{
    public static void main(String args []){
        Distance d1 = new Distance();
        d1.display();
        Distance d2 = new Distance(10,9);
        d2.display();
    }
    }
```

Output

```
The distance is : 1'2
The distance is : 10'9
```

Task 4 – Write a class Marks with three data members to store three marks. Create two constructors and a method to calculate and return the sum.

Source Code

```
class Marks{
    private int chemMarks;
    private int mathsMarks;
    private int phyMarks;
    public Marks(){
        chemMarks = 0;
        mathsMarks = 0;
        phyMarks = 0;
    }
    public Marks( int c, int m, int p){
        chemMarks = c;
        phyMarks = p;
        mathsMarks = m;
    }
    public int sum(){
     return chemMarks + phyMarks + mathsMarks;

     }
}
    public class RunnerLab2Task4{
        public static void main(String args []){
            Marks m1 = new Marks(75,48,90);
            System.out.println("The sum of marks is:" + m1.sum());
             Marks m2 = new Marks(90,80,70);
             System.out.println("The sum of marks is:" + m2.sum());
        }
    }
```

Output

```
The sum of marks is:213
The sum of marks is:240
```

Task 5 – Write a class Time with three data members to store hr, min and seconds. Create two constructors and apply checks to set valid time. Also create display function which displays all data members.

Source Code

```
class Time{
    private int hours;
    private int minutes;
    private int seconds;

    public Time(){
        hours = 9;
        minutes = 3;
        seconds = 2;
    }

    public Time(int h, int m, int s){

        if(h >= 0 && h <= 23){
            hours = h;
        }
        else{
            System.out.println("Invalid hours");
            hours = 0;
        }

        if(m >= 0 && m <= 59){
            minutes = m;
        }
        else{
            System.out.println("Invalid minutes");
            minutes = 0;
        }

        if(s >= 0 && s <= 59){
            seconds = s;
        }
        else{
            System.out.println("Invalid seconds");
            seconds = 0;
        }
    }
```

```
    public void display(){
        System.out.println("The time is: " + hours + ":" + minutes + ":" +
seconds);
    }
}

public class RunnerLab2Task5{
    public static void main(String [] args){
        Time t1 = new Time();
        t1.display();

        Time t2 = new Time(9,8,7);
        t2.display();
    }
}
```

Output

```
The time is: 9:3:2
The time is: 9:8:7
```

Lab 3- Passing and returning non primitive values from methods

Task 1 – Create a class — Distance‖ with two constructors (no argument, and two argument), two data members (feet and inches) . Create setter, getter and display method. Create a method that adds two Distance Objects and returns the added Distance Object.

Source Code

```
lass Distance {
    private double feet;
    private double inches;

    public Distance() {
        feet = 0.0;
        inches = 0.0;
    }

    public Distance(double f, double i) {
        feet = f;    // no conflict, so no 'this' needed
        inches = i;
    }

    public void setFeet(double f) {
        feet = f;
    }

    public void setInches(double i) {
        inches = i;
    }

    public double getFeet() {
        return feet;
    }

    public double getInches() {
        return inches;
    }

    // Add method without using 'this'
    public Distance add(Distance d) {
        double totalFeet = feet + d.getFeet();   // use different local variable
names
```

```
        double totalInches = inches + d.getInches();

        if(totalInches >= 12) {
            totalFeet += (int)(totalInches / 12); // convert extra inches to feet
            totalInches = totalInches % 12;       // remaining inches
        }

        return new Distance(totalFeet, totalInches);
    }

    public void display() {
        System.out.printf("The added distance is: %.2f feet and %.2f
inches.\n",  feet, inches);
    }
}

public class DistanceShow {
    public static void main(String[] args) {
        Distance d1 = new Distance(8.9, 6.9);
        Distance d2 = new Distance(2.0, 1.8);

        Distance d3 = d1.add(d2); // add distances

        d3.display(); // display result
    }
}
```

OutPut

```
C:\SEM2\OOPSLAB\Passing and returning non- primitive values from methods\LabTasks>java DistanceShow.java
The added distance is: 10.90 feet and 8.70 inches.
```

<u>Task 2 –</u> Create an Encapsulated class Book. Its data members are • author (String) • chapterNames[100] (String[]) Create two overloaded constructors, Create a method compareBooks, a method compareChapterNames, a runner class that declares two objects of type Book.

<u>Source Code</u>

```
class Book {

    private String author;
    private String[] chapterNames = new String[100];

    public Book() {
        author = "";
    }

    public Book(String a, String[] c) {
        author = a;
        chapterNames = c;
    }

    public void setAuthor(String a) {
        author = a;
    }

    public void setChapters(String[] c) {
        chapterNames = c;
    }

    public String getAuthor() {
        return author;
    }

    public static boolean compareBooks(Book b1, Book b2) {
        if (b1.author.equals(b2.author))
            return true;
        else
            return false;
    }

    public static Book compareChapterNames(Book b1, Book b2) {
        int count1 = 0;
```

```
        int count2 = 0;

        for (int i = 0; i < b1.chapterNames.length; i++) {
            if (b1.chapterNames[i] != null)
                count1++;
        }

        for (int i = 0; i < b2.chapterNames.length; i++) {
            if (b2.chapterNames[i] != null)
                count2++;
        }

        if (count1 > count2)
            return b1;
        else
            return b2;
    }
}

public class BookTesting {
    public static void main(String[] args) {

        Book book1 = new Book();
        book1.setAuthor("Ali");
        String[] ch1 = {"Intro", "Chapter1", "Chapter2"};
        book1.setChapters(ch1);

        String[] ch2 = {"Intro", "C1", "C2", "C3"};
        Book book2 = new Book("Ali", ch2);

        System.out.println(Book.compareBooks(book1, book2));

        Book bigger = Book.compareChapterNames(book1, book2);
        System.out.println(bigger.getAuthor());
    }
}
```

Output

```
true
Ali
```

<u>Task 3–</u> Define a class called Fraction. This class is used to represent a ratio of two integers. Create two constructors, set, get and display function. Include an additional method, equals, that takes as input another Fraction and returns true if the two fractions are identical and false if they are not.

<u>Source Code</u>

```
class Fraction {

    private int num;
    private int den;

    public Fraction() {
        num = 0;
        den = 1;
    }

    public Fraction(int n, int d) {
        num = n;
        den = d;
    }

    public void setNum(int n) {
        num = n;
    }

    public void setDen(int d) {
        den = d;
    }

    public int getNum() {
        return num;
    }

    public int getDen() {
        return den;
    }

    public void display() {
        System.out.println(num + "/" + den);
    }
```

```
    public boolean equals(Fraction f) {
        return (num == f.num && den == f.den);
    }
}

public class FractionTesting {
    public static void main(String[] args) {

        Fraction f1 = new Fraction(1, 2);
        Fraction f2 = new Fraction(1, 2);

        f1.display();
        f2.display();

        System.out.println(f1.equals(f2));
    }
}
```

Output

```
1/2
1/2
true
```

Project Topics Work:

Topic 1 –Digital Wallet

Part A – Design Draw class diagram that illustrates • Attributes • Methods

Digital Wallet
Attributes: - balance (double) - currency (string)
Methods: +DigitalWallet() + addMoney() : void + spendMoney() : void + showBalance() : void

Part B – Implementation • One class • One object in main • Use constructor

Source Code

```
import java.util.Scanner;

class DigitalWallet {
    private double balance;
    private String currency;
    public DigitalWallet() { //default constructor
        balance  = 0.0;
        currency = "PKR";
    }
    // Parameterized constructor
    public DigitalWallet(double b, String c) {
        balance  = b;
        currency = c;
    }
    public void addMoney() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter amount to add: ");
        double amount = sc.nextDouble();

        if (amount <= 0) {
            System.out.println("Invalid amount. Please enter a positive value.");
            return;
        }
```

```
        balance += amount;
        System.out.println("Money added! Current balance: " + currency + " " +
balance);
    }
    public void spendMoney() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter amount to spend: ");
        double amount = sc.nextDouble();
        if (amount <= 0) {
            System.out.println("Invalid amount.");
            return;
        }
        if (amount > balance) {
            System.out.println("Insufficient balance! Current balance: " +
currency + " " + balance);
        } else {
            balance -= amount;
            System.out.println("Spent " + currency + " " + amount +
                               ". Remaining balance: " + currency + " " + balance);
        }
    }
    public void showBalance() {
        System.out.println("Currency: " + currency);
        System.out.println("Balance : " + currency + " " + balance);
    }
}
public class DigitalWalletRunner {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        DigitalWallet wallet1 = new DigitalWallet();
        int choice = 0;
        while (choice != 4) {
            System.out.println("\n DIGITAL WALLET ");
            System.out.println("1. Add Money");
            System.out.println("2. Spend Money");
            System.out.println("3. Show Balance");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1: wallet1.addMoney();   break;
                case 2: wallet1.spendMoney(); break;
                case 3: wallet1.showBalance(); break;
                case 4: System.out.println("Exiting wallet. Goodbye!"); break;
```

```
                default: System.out.println("Invalid choice. Choose between 1 to
4 only.");
            }
        }
        sc.close();
    }
}
```

Output

```
 DIGITAL WALLET
1. Add Money
2. Spend Money
3. Show Balance
4. Exit
Enter choice: 1
Enter amount to add: 34
Money added! Current balance: PKR 34.0
```

```
 DIGITAL WALLET
1. Add Money
2. Spend Money
3. Show Balance
4. Exit
Enter choice:
2
Enter amount to spend: 31
Spent PKR 31.0. Remaining balance: PKR 3.0
```

```
 DIGITAL WALLET
1. Add Money
2. Spend Money
3. Show Balance
4. Exit
Enter choice: 3
Currency: PKR
Balance : PKR 3.0
```

Part C – Memory Analysis Answer the following questions

• How many objects exist in memory?

1 object exists in memory : wallet1.

• What changes inside object after each method call?

Method Called	Attribute Changed	Value after call
DigitalWallet	all initialized	Balance = 0.0, currency = pkr
addMoney()	balance	0.0 →5000.0
spendMoney()	balance	5000.0 → 3000.0
showBalance()	none	Only displays
exit	none	Program exits

Part D – Reflection • How does this object represent a real-world entity?

Real world digital wallets like jazzcash or easypaisa store two core piece of information. First is the balance and second is the currency. This simulated wallet represents these digital wallets and does actions like adding money, spending money and showing current balance like real world wallets do.

Topic 2 – Printer Machine

Part A – Design Draw class diagram that illustrates • Attributes • Methods.

Printer Machine
Attributes: -inkLevel(int) - paperCount (int) -isOn(boolean)
Methods: +PrinterMachine() +powerOn() : void +powerOff() : void +printPages() : void +refillInk() : void +addPaper() : void +showStatus() : void

Part B – Implementation • One class • One object in main • Use constructor

Source Code

```
import java.util.Scanner;
class PrinterMachine {
    private int inkLevel;
    private int paperCount;
    private boolean isOn;

    public PrinterMachine() { //defaultConstructor
        inkLevel    = 20;
        paperCount  = 10;
        isOn        = false;
    }

    public PrinterMachine(int inkLvl, int paperCnt, boolean powerStatus) {
//paramterezied constrctor
        inkLevel   = inkLvl;
        paperCount = paperCnt;
        isOn       = powerStatus;
    }

    public void powerOn() {  //question asked for power status checking so..
        if (isOn == false) {
            isOn = true;
```

```
            System.out.println("Printer is now ON.");
        } else {
            System.out.println("Printer is already ON.");
        }
    }

    public void powerOff() {
        if (isOn == true) {
            isOn = false;
            System.out.println("Printer is now OFF.");
        } else {
            System.out.println("Printer is already OFF.");
        }
    }

    public void printPages() {
    if (isOn == false) {
        System.out.println("Printer is OFF. Please turn it on first.");
        return;
    }
    if (inkLevel <= 0) {
        System.out.println("No ink! Please refill.");
        return;
    }
    if (paperCount <= 0) {
        System.out.println("No paper! Please add paper.");
        return;
    }

    Scanner sc = new Scanner(System.in);  // created locally inside method so
dont have to pass it

    System.out.print("Enter number of pages to print: ");
    int n = sc.nextInt();

    if (paperCount < n) {
        System.out.println("Not enough paper! Only " + paperCount + " sheets
left.");
        return;
    }
    if (inkLevel < n * 2) {
        System.out.println("Not enough ink to print " + n + " pages.");
        return;
    }
```

```
    System.out.println("Print on one side (1) or both sides (2)?");
    int sides = sc.nextInt();

    if (sides == 1) {
        inkLevel   -= n;
        paperCount -= n;
        System.out.println("Printed " + n + " page(s) single sided.");
    } else if (sides == 2) {
        inkLevel   -= n * 2;
        paperCount -= n;
        System.out.println("Printed " + n + " page(s) double sided.");
    } else {
        System.out.println("Invalid choice. Enter 1 or 2.");
    }
}
    public void refillInk() {
        if (inkLevel < 20) {
            System.out.println("Refilling ink...");
            inkLevel = 100;
            System.out.println("Ink is now full: " + inkLevel + "%");
        } else {
            System.out.println("Ink level is sufficient: " + inkLevel + "%");
        }
    }

    public void addPaper() {
        if (paperCount < 10) {
            System.out.println("Paper count is low. adding paper...");
            paperCount = 50;
            System.out.println("Paper count is now: " + paperCount);
        } else {
            System.out.println("Paper is sufficient: " + paperCount + "
sheets.");
        }
    }

    public void showStatus() {
        System.out.println("PRINTER STATUS");
        System.out.println("Ink Level   : " + inkLevel + "%");
        System.out.println("Paper Count : " + paperCount + " sheets");
        System.out.println("Power Status: " + (isOn ? "ON" : "OFF"));
    }
}

public class PrinterMachineRunner {
```

```java
    public static void main(String[] args) {

        PrinterMachine printer1 = new PrinterMachine();
        PrinterMachine printer2 = new PrinterMachine(80, 30, false);

        System.out.println("Printer 1: ");
        printer1.showStatus();
        printer1.powerOn();
        printer1.printPages();
        printer1.refillInk();
        printer1.addPaper();
        printer1.powerOff();
        printer1.showStatus();

        System.out.println("Printer 2: ");
        printer2.showStatus();
        printer2.powerOn();
        printer2.printPages();
        printer2.refillInk();
        printer2.addPaper();
        printer2.powerOff();
        printer2.showStatus();

    }
}
```

Output

```
Printer 1:
PRINTER STATUS
Ink Level   : 20%
Paper Count : 10 sheets
Power Status: OFF
Printer is now ON.
Enter number of pages to print: 2
Print on one side (1) or both sides (2)?
1
Printed 2 page(s) single sided.
Refilling ink...
Ink is now full: 100%
Paper count is low. adding paper...
Paper count is now: 50
Printer is now OFF.
```

```
PRINTER STATUS
Ink Level   : 100%
Paper Count : 50 sheets
Power Status: OFF
Printer 2:
PRINTER STATUS
Ink Level   : 80%
Paper Count : 30 sheets
Power Status: OFF
Printer is now ON.
Enter number of pages to print:
```

Part C – Memory Analysis Answer the following questions

• How many objects exist in memory?

2 objects exist in memory : printer1 and printer2

• What changes inside object after each method call?

Printer1		
Method Called	**Attributes Changed**	**Value After Call**
+PrinterMachine()	All initialized.	inkLevel = 20, paperCount = 10, isOn = false;
+ powerOn()	isOn	False → true
+printPages()	inkLevel, paperCount	20 → 15, 10→15
+refillInk	inkLevel	15 → 100
+ addPaper()	paperCount	5 → 50
+powerOff()	isOn	True → false
+showStatus()	none	Only display output

Printer2		
Method Called	**Attributes Changed**	**Value After Call**
+PrinterMachine()	All initialized.	inkLevel = 80, paperCount = 30, isOn = false;
+ powerOn()	isOn	False → true
+printPages()	inkLevel, paperCount	80 → 70, 30 → 25
+refillInk	inkLevel	70 % is enough. No change
+ addPaper()	paperCount	25 sheets enough. No chnage
+powerOff()	isOn	True → false
+showStatus()	none	Only display output

Part D – Reflection • How does this object represent a real-world entity?

This printer machine simulates a real world printer. In a real printer, you check the ink level, the pages in the tray and whether it is powered on or not. The same is being checked here through the program. The methods represent the real world actions we do with pronters like turning them on or off, refilling ink, adding paper, printing documents etc.

Topic 3 –Smart Device Simulator

Part A – Design Draw class diagram that illustrates • Attributes • Methods

Smart Device(Class)
Attributes: - brand(string) - batteryLevel(int) - volume (int) - isOn (boolean)
Methods: +SmartDevice() + powerOn() : void + powerOff() : void + increaseVolume() : void + useApp() : void +charge() : void +showStatus() : void

Part B – Implementation • One class • One object in main • Use constructor

Source Code

```
import java.util.Scanner;
class SmartDevice{
    private String brand;
    private int batteryLevel;
    private int volume;
    private boolean isOn;
    String [] apps = { "Settings", "Instagram" , "Snapchat", "Youtube" ,
"FarmGame"};

     public SmartDevice(){    //default constrctor
        brand = "nokia";
        volume = 10;
        isOn = false;
        batteryLevel = 10;
    }

    public SmartDevice(String b, int vo, int batteryLvl, boolean on) {
        brand = b;
        volume = (vo <= 20) ? vo : 20;
```

```
        batteryLevel = (batteryLvl <= 100) ? batteryLvl : 100;
        isOn = on;
    }

    public void powerOn() {
        if(isOn == false){
        isOn = true;
        System.out.println("The device is now ON.");
        } else {
        System.out.println("Device is already ON.");
        }
    }
    public void powerOff() {
        if(isOn == true){
        isOn = false;
        System.out.println("The device is now OFF.");
        } else {
        System.out.println("Device is already OFF.");
        }
    }
    public void increaseVolume() {
        if( isOn== false) {
            System.out.println("DEVICE IS OFF!");
            return;
        }
        if(volume < 20){
            volume++;
            System.out.println("Volume increased to: " + volume);

        }
    }
    public void useApp() {
    if (isOn == false) {
        System.out.println("DEVICE IS OFF!");
        return;
    }

    Scanner sc = new Scanner(System.in);

    System.out.println("Select the app you want to use:");
    for (int i = 0; i < apps.length; i++) {
        System.out.println((i + 1) + ". " + apps[i]);
    }

    System.out.print("Enter app number: ");
```

```
    int choice = sc.nextInt();

    if (choice >= 1 && choice <= apps.length) {
        System.out.println("Opening " + apps[choice - 1] + "...");
    } else {
        System.out.println("Invalid choice!");
    }
}
    public void charge() {
       if( batteryLevel > 50){
        System.out.println( "Battery is: " + batteryLevel + "%");
    }
       else if ( batteryLevel > 20 && batteryLevel <= 50){
        System.out.println("Battery is: " + batteryLevel + "%");
        System.out.println("Battery is at a moderate level. You can continue
using the device.");
    }
        else if( batteryLevel < 20){
        System.out.println("LOW BATTERY" + "\n Kindly connect a charger.");
        batteryLevel = 100;
        System.out.println("Battery is now fully charged: " + batteryLevel +
"%");
    }
        else
        {
            System.out.println("Battery is: " + batteryLevel + "%");
            System.out.println("Put your device on charging.");
        }
    }

    public void showStatus() {
        System.out.println("Brand: " + brand);
        System.out.println("Battery Level: " + batteryLevel + "%");
        System.out.println("Volume: " + volume);
        System.out.println("Power Status: " + (isOn ? "On" : "Off"));
    }
}

    public class SmartDeviceRunner{
        public static void main(String [] args){
            SmartDevice phone1 = new SmartDevice ();
            phone1.showStatus();
            phone1.charge();
            phone1.powerOn();
            phone1.increaseVolume();
```

```
        phone1.useApp();
        SmartDevice phone2 = new SmartDevice ("GooglePixel",30,10, true);
        phone2.showStatus();
        phone2.charge();
        phone2.powerOn();
        phone2.increaseVolume();
        phone2.useApp();


    }
}
```

Output

```
Brand: nokia
Battery Level: 10%
Volume: 10
Power Status: Off
LOW BATTERY
 Kindly connect a charger.
Battery is now fully charged: 100%
The device is now ON.
Volume increased to: 11
Select the app you want to use:
1. Settings
2. Instagram
3. Snapchat
4. Youtube
5. FarmGame
Enter app number:
```

```
Enter app number: 3
Opening Snapchat...
Brand: GooglePixel
Battery Level: 10%
Volume: 20
Power Status: On
LOW BATTERY
 Kindly connect a charger.
Battery is now fully charged: 100%
Device is already ON.
Select the app you want to use:
1. Settings
2. Instagram
3. Snapchat
4. Youtube
5. FarmGame
Enter app number:
```

Part C – Memory Analysis Answer the following questions

• How many objects exist in memory?

2 objects exist in the memory: phone1 and phone2.

• What changes inside object after each method call?

Phone1		
Method Called	**Attributes Changed**	**Value After Call**
+SmartDevice()	Brand, batteryLevel, volume, isOn. All initialized.	Brand = nokia, batteryLevel = 10, volume = 10, isOn = false
+charge()	batteryLevel	10 → 100
+powerOn()	isOn	False →true

+powerOff()	isOn	True → false
+ increaseVolume()	volume	10 → 11
+useApp()	none	Only displays output
+showStatus()	none	Only display output

Phone2		
Method Called	**Attributes Changed**	**Value After Call**
+SmartDevice()	Brand, batteryLevel, volume, isOn. All initialized.	Brand = Google pixel, batteryLevel = 10, volume = 15, isOn = false
+charge()	batteryLevel	10 → 100
+powerOn()	isOn	False →true
+powerOff()	isOn	True → false
+increaseVolume()	volume	15→ 16
+useApp()	none	Only displays output
+showStatus()	none	Only display output

Part D – Reflection • How does this object represent a real-world entity?

This smart device is a perfect representation of a real world mobile phone. Like a mobile phone, it has the ability to adjust its volume, use different apps. It has battery feature too and if battery is below the optimal value, the device is put on charging mode. Additionally, it also has the feature of powering the cell phone, turning it off or on.

Topic 4 –Traffic Light

Traffic Light

Part A – Design Draw class diagram that illustrates • Attributes • Methods

TrafficLight
Attributes: - Color(string) - Timer(int)
Methods: + changeColor() : void + resetTimer() : void + showStatus() : void

Part B – Implementation • One class • One object in main • Use constructor

Source Code

```
class TrafficLight{

private String color;
private int timer;

public TrafficLight(){
    color = "Red";
    timer = 60;
}
public TrafficLight(String c, int t){
    color = c;
    timer = t;
}
public void changeColor() {
        if (color.equals("Red")) {
            color = "Green";
            timer = 45;
            System.out.println("Light changed to GREEN. Timer: " + timer + "s");
        } else if (color.equals("Green")) {
            color = "Yellow";
            timer = 10;
            System.out.println("Light changed to YELLOW. Timer: " + timer + "s");
```

```
        } else if (color.equals("Yellow")) {
            color = "Red";
            timer = 60;
            System.out.println("Light changed to RED. Timer: " + timer + "s");
        }
    }

    public void resetTimer() {
        if (color.equals("Red")) {
            timer = 60;
        } else if (color.equals("Green")) {
            timer = 45;
        } else if (color.equals("Yellow")) {
            timer = 10;
        }
        System.out.println("Timer reset to " + timer + "s for " + color + "
light.");
    }

    public void showStatus() {
        System.out.println("TRAFFIC LIGHT STATUS");
        System.out.println("Current Color : " + color);
        System.out.println("Timer         : " + timer + " seconds");
    }
}

public class TrafficLightRunner {
    public static void main(String[] args) {

        TrafficLight light = new TrafficLight();

        light.showStatus();

        light.changeColor();
        light.showStatus();

        light.changeColor();
        light.showStatus();

        light.changeColor();

        light.resetTimer();
        light.showStatus();
    }
}
```

[redacted]

Output

```
TRAFFIC LIGHT STATUS
Current Color : Red
Timer         : 60 seconds
Light changed to GREEN. Timer: 45s
TRAFFIC LIGHT STATUS
Current Color : Green
Timer         : 45 seconds
Light changed to YELLOW. Timer: 10s
TRAFFIC LIGHT STATUS
Current Color : Yellow
Timer         : 10 seconds
Light changed to RED. Timer: 60s
Timer reset to 60s for Red light.
TRAFFIC LIGHT STATUS
Current Color : Red
Timer         : 60 seconds
```

Part C – Memory Analysis Answer the following questions

• How many objects exist in memory?

1 object exists in memory : light**.**

• What changes inside object after each method call?

Method Called	**Attribute Changed**	**Changed Value**
TrafficLight()	All initilaized	Color = red, timer = 60
changeColor()	Color, timer	Red, 60 → green, 45
changeColor()	Color, timer	Green, 45 → yellow, 10
changeColor()	Color, timer	Yellow, 10 → red, 60
resetTimer()	Color, timer	60 → 60(already default)
showStatus	none	Only displays. Nothing chnages

Part D – Reflection • How does this object represent a real-world entity?

Real life traffic lights change colors after a set time that is already.